

NAMMA ROBOTICS

presents

Arduino Projects for Beginners

*30+ Hands-On Projects with Full Source Code
& Step-by-Step Circuit Diagrams*

[@namma_robotics.in](https://namma_robotics.in)

Arduino Projects for Beginners

Copyright © 2026 Namma Robotics

Instagram / Social: @namma_robotics.in

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

This eBook is intended for personal, educational, and non-commercial use only. Unauthorized resale, redistribution, or modification of this work is strictly prohibited and may result in legal action.

For permissions, licensing, bulk orders, or collaborations, please contact us at namma_robotics.in.

Trademarks

Arduino® is a registered trademark of Arduino SA. All other product names and company names mentioned in this book are the property of their respective owners. Namma Robotics is not affiliated with Arduino SA or any other company mentioned herein.

Printed and distributed by Namma Robotics • @namma_robotics.in • 2026

Table of Contents

Introduction.....	5
Safety & Disclaimer.....	6
What You Need to Get Started.....	7

BEGINNER PROJECTS

1. Blinking LED	9
2. Traffic Light System	10
3. Pushbutton LED Toggle	11
4. Potentiometer Brightness	12
5. LDR Night Light	13
6. RGB LED Colour Mixer	14
7. Servo Motor Control	15
8. Ultrasonic Distance Meter	16
9. LCD Display Hello World	17
10. Temperature Monitor (LM35)	18
11. Buzzer Melody Player	19

INTERMEDIATE PROJECTS

12. PIR Motion Detector Alarm	20
13. DHT11 Weather Station	21
14. Keypad Password System	22
15. RFID Card Access Control	23
16. Bluetooth LED Control	24
17. Stepper Motor Control	25
18. Soil Moisture Auto-Watering	26
19. DC Motor Speed & Direction	27
20. IR Remote Control System	28
21. OLED Mini Display	29
22. Smoke & Gas Detector	30
23. 7-Segment Display Counter	31
24. Sound Level Meter	32

25. Laser Tripwire Security	33
26. Real-Time Clock Calendar	34
27. Joystick Servo Controller	35
28. Water Level Alarm	36
29. Digital Dice	37

ADVANCED PROJECT

30. Line Following Robot	38
--------------------------------	----

Thank You & Resources.....	40
----------------------------	----

Introduction

Welcome to Arduino Projects for Beginners, brought to you by Namma Robotics — your trusted companion on the journey from complete beginner to confident maker.

Whether you are a student curious about electronics, a hobbyist who always wanted to build something that moves, blinks, or responds to the world around it, or a teacher looking for hands-on STEM activities — this eBook is written for you.

What is Arduino?

Arduino is an open-source electronics platform combining easy-to-use hardware and software. At its heart is a microcontroller — a tiny computer on a chip — that you program using the Arduino IDE (Integrated Development Environment). The Arduino Uno, this eBook's primary board, uses the ATmega328P microcontroller running at 16 MHz with 32KB of flash memory.

Unlike desktop computers, an Arduino reacts to the physical world in real time: it reads sensors (temperature, light, distance), controls actuators (motors, LEDs, displays), and communicates wirelessly — all within a few seconds of pressing Upload.

How This Book is Structured

Each project in this eBook follows a consistent format to make learning as easy as possible:

- Difficulty Level & Components — everything you need listed upfront
- Circuit Diagram — ASCII-art wiring guide you can reference while building
- How It Works — the science and engineering explained simply
- Full Source Code — copy-paste ready, heavily commented Arduino sketches
- Pro Tips — common mistakes to avoid and ideas to extend the project

Learning Path

Projects 1-11 are beginner level, requiring no prior experience. Projects 12-29 are intermediate, introducing libraries, communication protocols (I2C, SPI, UART), and modules. Project 30 is an advanced culmination project — a fully autonomous robot.

Namma Robotics Tip

Don't just copy the code — read the comments, change a value, and observe what happens.

Breaking things is how you learn. That's why components are inexpensive!

Join our community at namma_robotics.in for video tutorials and support.

Safety & Disclaimer

IMPORTANT — Please Read Before Starting

This eBook contains instructions for working with electronic components and electricity. Always follow safety precautions. Failure to do so may result in injury or property damage. Adult supervision is recommended for users under 14 years of age.

General Safety Rules

- Always disconnect power before changing wiring or components.
- Double-check all connections against the circuit diagram before powering on.
- Never short-circuit the power supply — this can damage components or cause burns.
- Keep liquids away from all electronics at all times.
- Use the correct power supply voltage — Arduino Uno requires 5V USB or 7-12V DC barrel.
- If a component gets unusually hot or smells of burning, immediately disconnect power.
- Laser modules (Project 25) are CLASS IIA — never point at eyes.
- Relay and motor circuits can switch higher voltages — ensure all mains voltage work is done by a qualified electrician.

Limitation of Liability

Namma Robotics and the authors of this eBook have made every effort to ensure accuracy of the information and code presented. However, we make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability with respect to the information, source code, or circuits contained in this eBook.

Any reliance you place on such information is strictly at your own risk. In no event will Namma Robotics be liable for any loss or damage, including without limitation, indirect or consequential loss or damage, arising out of or in connection with the use of this eBook.

The source code provided is intended for educational purposes. It is your responsibility to ensure any project built from this eBook meets applicable safety and regulatory standards before use in any real-world application.

Component Safety Notes

Component	Safety Note
LEDs	Always use current-limiting resistors (minimum 100Ω). Direct connection to 5V will burn out the LED.
Motors	Use driver boards (L298N, ULN2003) — never connect motors directly to Arduino pins.
Relays	Relay modules can switch mains voltage. Only qualified electricians should work with 230V/110V circuits.
Lithium batteries	Never short-circuit, overcharge, or puncture Li-Po/Li-Ion cells. Use a protection circuit.
Gas sensors (MQ-series)	These sensors have a heater element that gets very hot. Do not touch the metal mesh.
Laser modules	Never look directly into a laser beam or aim at people or animals.

What You Need to Get Started

Before diving into projects, let's set up your workspace. Here is everything you need to get started with Arduino development.

Essential Hardware

Item	Purpose	Approx Cost
Arduino Uno R3	Main microcontroller board	₹400 / \$5
USB-A to USB-B Cable	Connect Arduino to PC	₹80 / \$1
Breadboard (830 points)	Prototype circuits without soldering	₹100 / \$2
Jumper Wire Kit	Connect components	₹150 / \$2
LED Assortment (R/G/B/Y)	Output indicator in most projects	₹100 / \$1
Resistor Kit (220Ω, 1kΩ, 10kΩ)	Current limiting and pull-ups	₹100 / \$2
Pushbutton x5	Digital input in many projects	₹50 / \$1
Potentiometer 10kΩ x3	Analog input / contrast control	₹50 / \$1
HC-SR04 Ultrasonic Sensor	Distance measurement	₹120 / \$2
SG90 Servo Motor	Positional motor control	₹120 / \$2
DHT11 Sensor	Temperature & humidity	₹80 / \$1
16x2 LCD Display	Text output display	₹150 / \$3

Software Setup (Free)

1. Step 1 Download Arduino IDE from arduino.cc/en/software — available for Windows, macOS, and Linux.
2. Step 2 Install the IDE. On Windows, also install the CH340 USB driver if your Arduino Uno is a clone.
3. Step 3 Open Arduino IDE > Tools > Board > Arduino AVR Boards > Arduino Uno.
4. Step 4 Open Arduino IDE > Tools > Port > Select the COM port (Windows) or /dev/ttyUSB0 (Linux).
5. Step 5 Open File > Examples > Basics > Blink and click Upload (arrow button). Your onboard LED should blink!

Installing Libraries

Many projects require external libraries. Install them via Sketch > Include Library > Manage Libraries in the Arduino IDE. Search for the library name listed at the top of each project's source code. Click Install.

Recommended Starter Kit

All components used in this eBook are available as a single bundle on namma_robotics.in. The Namma Robotics Beginner Arduino Kit includes all 30+ project components pre-selected and tested — ready to build!

Pro Workspace Setup

Use an anti-static mat to protect sensitive components from electrostatic discharge (ESD).

Keep small components in labelled zip-lock bags or a component organizer box.

Always work on a non-conductive surface — not directly on a metal desk.

A multimeter (DMM) is the most valuable tool you can add to your kit. (~₹300 / \$5)

Project 1 Blinking LED

Difficulty: **Beginner** | Category: **Output**

Description

The classic first Arduino project. Learn to turn an LED on and off at regular intervals using digital output and the delay () function. This foundational project teaches you how to write a basic Arduino sketch and how current-limiting resistors protect LEDs.

Components Required

Arduino Uno, LED (any color), 220Ω resistor, breadboard, jumper wires

Circuit Diagram

```
+-----+
|  CIRCUIT DIAGRAM  |
|  Blinking LED      |
+-----+
|                    |
| Arduino Pin 13 --[220R]--|>|-- GND
|           resistor LED |
|                    |
| Pin 13 -> Anode (+) of LED |
| GND    -> Cathode (-) of LED |
|      (via 220Ω resistor)    |
+-----+
```

How It Works

The Arduino's pin 13 is set as OUTPUT. In the loop, it toggles HIGH (5V) then LOW (0V) with 1-second delays. Current flows through the resistor and LED when the pin is HIGH, limiting current to ~10mA to protect the LED.

Source Code

```
// Project 1: Blinking LED
// @namma_robotics.in

int ledPin = 13; // LED connected to digital pin 13

void setup() {
  pinMode(ledPin, OUTPUT); // Set pin as output
}

void loop() {
  digitalWrite(ledPin, HIGH); // Turn LED ON (5V)
```

```
delay(1000);          // Wait 1000 ms = 1 second  
digitalWrite(ledPin, LOW); // Turn LED OFF (0V)  
delay(1000);          // Wait 1 second  
}
```

Pro Tips

Try changing the delay values to make the LED blink faster or slower

The built-in LED on most Arduinos is also on pin 13 — no external LED needed to test!

Project 2 Traffic Light System

Difficulty: **Beginner** | Category: **Output**

Description

Simulate a realistic traffic light sequence using three colored LEDs. This project reinforces multi-pin output control and timing logic. You will learn how to sequence actions and understand real-world embedded timing.

Components Required

Arduino Uno, Red/Yellow/Green LEDs (1 each), 3x 220Ω resistors, breadboard, wires

Circuit Diagram

```
+-----+
| TRAFFIC LIGHT DIAGRAM |
+-----+
|           |
| Pin 11 --[220R]--|>|-- GND (RED LED) |
| Pin 10 --[220R]--|>|-- GND (YELLOW LED) |
| Pin 9  --[220R]--|>|-- GND (GREEN LED) |
|           |
| All cathodes (-) -> GND |
+-----+
```

How It Works

Each LED is connected to a separate digital pin through a 220Ω resistor. The program cycles through Green (go) → Yellow (caution) → Red (stop) in a timed sequence that mimics a real traffic light.

Source Code

```
// Project 2: Traffic Light
// @namma_robotics.in

int red = 11, yellow = 10, green = 9;

void setup() {
  pinMode(red, OUTPUT);
  pinMode(yellow, OUTPUT);
  pinMode(green, OUTPUT);
}

void loop() {
  // Green phase
  digitalWrite(green, HIGH);
  delay(4000);
```

```
digitalWrite(green, LOW);

// Yellow phase
digitalWrite(yellow, HIGH);
delay(1500);
digitalWrite(yellow, LOW);

// Red phase
digitalWrite(red, HIGH);
delay(4000);
digitalWrite(red, LOW);
}
```

Pro Tips

Add a pedestrian button (pushbutton + INPUT_PULLUP) to request a crossing signal

Use millis() instead of delay() for non-blocking timing in advanced versions

Project 3 Pushbutton LED Toggle

Difficulty: **Beginner** | Category: [Input/Output](#)

Description

Read a pushbutton and toggle an LED on every press. Introduces digital INPUT, state tracking, and basic debouncing to prevent false triggers from electrical noise.

Components Required

Arduino Uno, pushbutton, LED, 220Ω resistor, 10kΩ resistor (optional), breadboard

Circuit Diagram

```
+-----+
|  BUTTON LED CIRCUIT  |
+-----+
|           |
| 5V -> Button -> Pin 2 |
| Pin 2 also -> 10kR -> GND (pull-down)
|           |
| Pin 13 -> 220R -> LED -> GND
+-----+
```

How It Works

The button connects 5V to pin 2 when pressed. A 10kΩ pull-down resistor ensures the pin reads LOW when button is released. The sketch detects a LOW→HIGH transition (rising edge) and flips the LED state.

Source Code

```
// Project 3: Pushbutton LED Toggle
// @namma_robotics.in

int btnPin = 2, ledPin = 13;
int ledState = LOW;
int lastBtnState = LOW;

void setup() {
  pinMode(btnPin, INPUT);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int btnState = digitalRead(btnPin);

  // Detect rising edge (button just pressed)
```

```
if (btnState == HIGH && lastBtnState == LOW) {  
  ledState = !ledState;      // Toggle LED  
  digitalWrite(ledPin, ledState);  
  delay(50);                 // Simple debounce  
}  
  
lastBtnState = btnState;  
}
```

Pro Tips

Use INPUT_PULLUP mode and wire the button to GND instead, to skip the external pull-down resistor

For robust debouncing, use the Bounce2 library

Project 4 Potentiometer Brightness Control

Difficulty: **Beginner** | Category: **Analog**

Description

Control LED brightness smoothly using a potentiometer. Learn analog input reading (0–1023) and PWM output (0–255) with map() function.

Components Required

Arduino Uno, potentiometer (10kΩ), LED, 220Ω resistor, breadboard

Circuit Diagram

```
+-----+
| POTENTIOMETER CIRCUIT |
+-----+
|           |
| Pot Left -> GND      |
| Pot Middle -> A0 (wiper) |
| Pot Right -> 5V      |
|           |
| Pin 9 (PWM) -> 220R -> LED -> GND
+-----+
```

How It Works

The potentiometer forms a voltage divider. Its middle pin (wiper) outputs 0–5V depending on rotation, which analogRead() converts to 0–1023. The map() function scales this to 0–255 for analogWrite() PWM brightness.

Source Code

```
// Project 4: Potentiometer LED Brightness
// @namma_robotics.in

int potPin = A0; // Potentiometer wiper
int ledPin = 9;  // PWM-capable pin

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int potValue = analogRead(potPin); // Read 0-1023
  int brightness = map(potValue, 0, 1023, 0, 255); // Scale to PWM
  analogWrite(ledPin, brightness); // Set brightness
```

```
Serial.print("Pot: "); Serial.print(potValue);  
Serial.print(" Brightness: "); Serial.println(brightness);  
delay(50);  
}
```

Pro Tips

PWM pins on Arduino Uno are: 3, 5, 6, 9, 10, 11

Try connecting an RGB LED and controlling each color channel with a separate potentiometer

Project 5 LDR Night Light

Difficulty: **Beginner** | Category: **Sensor**

Description

Automatically turn on an LED when the room gets dark using a Light Dependent Resistor. A great introduction to sensors and threshold-based logic.

Components Required

Arduino Uno, LDR (Light Dependent Resistor), 10kΩ resistor, LED, 220Ω resistor

Circuit Diagram

```
+-----+
| LDR VOLTAGE DIVIDER |
+-----+
|           |
| 5V -> LDR -> A0 (junction)|
|   A0 -> 10kR -> GND|
|           |
| Pin 13 -> 220R -> LED -> GND
+-----+
```

How It Works

The LDR and 10kΩ resistor form a voltage divider. In bright light, LDR resistance is low, so A0 reads high. In darkness, LDR resistance rises, pulling A0 lower. The sketch compares the reading to a threshold.

Source Code

```
// Project 5: LDR Night Light
// @namma_robotics.in

int ldrPin = A0;
int ledPin = 13;
int threshold = 400; // Adjust based on your lighting

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int light = analogRead(ldrPin);
  Serial.print("Light level: "); Serial.println(light);
}
```

```
if (light < threshold) {  
  digitalWrite(ledPin, HIGH); // Dark -> LED ON  
} else {  
  digitalWrite(ledPin, LOW); // Bright -> LED OFF  
}  
delay(200);  
}
```

Pro Tips

Print the serial monitor output to calibrate your threshold value

Replace the LED with a relay module to control a real lamp

Project 6 RGB LED Color Mixer

Difficulty: **Beginner** | Category: **Output**

Description

Control an RGB LED to produce any color by mixing Red, Green, and Blue channels using PWM. Understand additive color mixing and multi-channel PWM control.

Components Required

Arduino Uno, common-cathode RGB LED, 3x 220Ω resistors, breadboard

Circuit Diagram

```
+-----+
| RGB LED CIRCUIT |
+-----+
|               |
| Pin 9 -> 220R -> R pin |
| Pin 10 -> 220R -> G pin |
| Pin 11 -> 220R -> B pin |
| Common cathode -> GND |
|               |
| (Common anode: flip to 5V and invert logic)
+-----+
```

How It Works

Three PWM pins independently control the Red, Green, and Blue elements inside the RGB LED. Mixing different intensities (0–255) produces millions of color combinations through additive color mixing.

Source Code

```
// Project 6: RGB LED Color Mixer
// @namma_robotics.in

int rPin = 9, gPin = 10, bPin = 11;

void setColor(int r, int g, int b) {
  analogWrite(rPin, r);
  analogWrite(gPin, g);
  analogWrite(bPin, b);
}

void setup() {
  // Pins are automatically OUTPUT when using analogWrite
}
```

```
void loop() {  
  setColor(255, 0, 0); delay(800); // Red  
  setColor( 0, 255, 0); delay(800); // Green  
  setColor( 0, 0, 255); delay(800); // Blue  
  setColor(255, 255, 0); delay(800); // Yellow  
  setColor( 0, 255, 255); delay(800); // Cyan  
  setColor(255, 0, 255); delay(800); // Magenta  
  setColor(255, 255, 255); delay(800); // White  
  setColor(128, 0, 255); delay(800); // Purple  
}
```

Pro Tips

Use three potentiometers (A0, A1, A2) to make an interactive color mixer

Try smooth color fading with a for-loop incrementing/decrementing values

Project 7 Servo Motor Control

Difficulty: **Beginner** | Category: **Motor**

Description

Control a servo motor to sweep from 0° to 180° using the built-in Servo library. Servo motors are used in robotics, camera gimbals, and automation projects.

Components Required

Arduino Uno, SG90 micro servo motor, jumper wires

Circuit Diagram

```
+-----+
| SERVO MOTOR CIRCUIT |
+-----+
|           |
| Servo BROWN -> GND   |
| Servo RED    -> 5V    |
| Servo ORANGE -> Pin 9  |
|           |
| Note: For multiple servos|
| use external 5V supply  |
+-----+
```

How It Works

The Servo library generates precise PWM pulses (1ms-2ms width) on the signal pin. The servo motor's internal controller reads this pulse width and moves its shaft to the corresponding angle.

Source Code

```
// Project 7: Servo Motor Sweep
// @namma_robotics.in

#include <Servo.h>

Servo myServo;
int pos = 0;

void setup() {
  myServo.attach(9); // Servo signal pin
  Serial.begin(9600);
}

void loop() {
  // Sweep 0 to 180 degrees
```

```
for (pos = 0; pos <= 180; pos++) {  
  myServo.write(pos);  
  Serial.println(pos);  
  delay(15);  
}  
// Sweep back 180 to 0 degrees  
for (pos = 180; pos >= 0; pos--) {  
  myServo.write(pos);  
  delay(15);  
}  
}
```

Pro Tips

Control angle with a potentiometer: `myServo.write(map(analogRead(A0), 0, 1023, 0, 180))`

SG90 operates at 4.8–6V; larger MG996R servos need external power

Project 8 Ultrasonic Distance Meter

Difficulty: **Beginner** | Category: **Sensor**

Description

Measure distances from 2cm to 400cm using ultrasonic sound pulses. The HC-SR04 is the foundation for parking sensors, robots, and level meters.

Components Required

Arduino Uno, HC-SR04 ultrasonic sensor, jumper wires, (optional: LCD or LEDs)

Circuit Diagram

```
+-----+
| HC-SR04 CIRCUIT |
+-----+
|               |
| HC-SR04 VCC -> 5V   |
| HC-SR04 GND -> GND  |
| HC-SR04 TRIG -> Pin 9 |
| HC-SR04 ECHO -> Pin 10 |
+-----+
```

How It Works

A 10µs HIGH pulse on TRIG fires an ultrasonic burst (8 × 40kHz pulses). The ECHO pin goes HIGH for the duration the sound takes to travel and return. Distance = (pulse duration × 0.034 cm/µs) / 2.

Source Code

```
// Project 8: Ultrasonic Distance Meter
// @namma_robotics.in

int trigPin = 9, echoPin = 10;

void setup() {
  Serial.begin(9600);
  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);
}

float getDistance() {
  // Send ultrasonic pulse
  digitalWrite(trigPin, LOW); delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
```

```
// Measure echo duration
long duration = pulseIn(echoPin, HIGH);

// Convert to cm (speed of sound = 0.034 cm/μs, divide by 2 for round trip)
return duration * 0.034 / 2.0;
}

void loop() {
  float dist = getDistance();
  Serial.print("Distance: ");
  Serial.print(dist);
  Serial.println(" cm");
  delay(500);
}
```

Pro Tips

Avoid measuring at angles greater than 15° from perpendicular to target

Minimum reliable range is ~2cm; below this the echo overlaps the trigger pulse

Project 9 LCD Display Hello World

Difficulty: **Beginner** | Category: **Display**

Description

Display text on a 16x2 character LCD in 4-bit parallel mode. Learn the LiquidCrystal library, cursor positioning, and contrast adjustment.

Components Required

Arduino Uno, 16x2 LCD, 10kΩ potentiometer, jumper wires

Circuit Diagram

```
+-----+
| 16x2 LCD CIRCUIT |
+-----+
|                 |
| LCD Pin 1 (VSS) -> GND |
| LCD Pin 2 (VDD) -> 5V |
| LCD Pin 3 (V0) -> Pot wiper (contrast)
| LCD Pin 4 (RS) -> Pin 12 |
| LCD Pin 5 (R/W) -> GND |
| LCD Pin 6 (EN) -> Pin 11 |
| LCD Pin 11 (D4) -> Pin 5 |
| LCD Pin 12 (D5) -> Pin 4 |
| LCD Pin 13 (D6) -> Pin 3 |
| LCD Pin 14 (D7) -> Pin 2 |
| LCD Pin 15 (A+) -> 5V (via 220R)
| LCD Pin 16 (K) -> GND |
+-----+
```

How It Works

The LiquidCrystal library communicates with the LCD controller (HD44780) in 4-bit mode, reducing the pin count. The potentiometer adjusts the contrast voltage (V0 pin), and the backlight LED is always on.

Source Code

```
// Project 9: LCD Display
// @namma_robotics.in

#include <LiquidCrystal.h>

// RS, EN, D4, D5, D6, D7
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
void setup() {  
  lcd.begin(16, 2);    // 16 columns, 2 rows  
  
  lcd.setCursor(0, 0);  // Column 0, Row 0  
  lcd.print("Namma Robotics");  
  
  lcd.setCursor(0, 1);  // Column 0, Row 1  
  lcd.print("Arduino Rocks!");  
}  
  
void loop() {  
  // Scroll text left across the display  
  lcd.scrollDisplayLeft();  
  delay(400);  
}
```

Pro Tips

If screen shows blank or all blocks, adjust the contrast potentiometer
Use `lcd.clear()` to erase the display, `lcd.noCursor()` to hide the cursor

Project 10 Temperature Monitor (LM35)

Difficulty: **Beginner** | Category: **Sensor**

Description

Read room temperature from the precision LM35 analog sensor. Display Celsius and Fahrenheit readings and learn sensor calibration.

Components Required

Arduino Uno, LM35 temperature sensor, LCD or Serial Monitor

Circuit Diagram

```
+-----+
| LM35 CIRCUIT      |
+-----+
|                   |
| LM35 Pin 1 (VCC) -> 5V |
| LM35 Pin 2 (Vout)-> A0 |
| LM35 Pin 3 (GND) -> GND |
|                   |
| Note: Flat face of LM35 |
| should face you when    |
| identifying pins        |
+-----+
```

How It Works

The LM35 outputs 10mV per degree Celsius. At 25°C it outputs 250mV. Arduino reads 0–1023 over 0–5V range. Formula: $\text{Temp}(\text{°C}) = (\text{analogRead} \times 5.0 / 1024.0) \times 100$.

Source Code

```
// Project 10: LM35 Temperature Sensor
// @namma_robotics.in

int sensorPin = A0;

void setup() {
  Serial.begin(9600);
  Serial.println("Temperature Monitor Ready");
}

void loop() {
  int rawValue = analogRead(sensorPin);

  // Convert raw reading to voltage (5V reference)
```

```
float voltage = rawValue * (5.0 / 1024.0);

// LM35 outputs 10mV per degree Celsius
float tempC = voltage * 100.0;
float tempF = (tempC * 9.0 / 5.0) + 32.0;

Serial.print("Temp: ");
Serial.print(tempC, 1);
Serial.print(" C / ");
Serial.print(tempF, 1);
Serial.println(" F");

delay(1000);
}
```

Pro Tips

For higher accuracy, use `analogReference(INTERNAL)` with the 1.1V internal reference
The DHT11/DHT22 sensors add humidity readings at slightly higher cost

Project 11 Buzzer Melody Player

Difficulty: **Beginner** | Category: **Audio**

Description

Play a recognizable melody (Twinkle Twinkle Little Star) using the tone() function. Learn musical note frequencies and timing in code.

Components Required

Arduino Uno, passive buzzer (NOT active buzzer), jumper wire

Circuit Diagram

```
+-----+
| PASSIVE BUZZER CIRCUIT |
+-----+
|           |
| Pin 8 -> Buzzer(+) pin |
| GND  -> Buzzer(-) pin |
|           |
| IMPORTANT: Use PASSIVE |
| buzzer (no built-in osc.)|
| Active buzzers don't work|
+-----+
```

How It Works

tone(pin, frequency, duration) generates a square wave at the given frequency on the pin. The passive buzzer's membrane vibrates at the frequency, producing sound. Musical notes have specific frequencies (A4 = 440Hz).

Source Code

```
// Project 11: Buzzer Melody Player
// Twinkle Twinkle Little Star
// @namma_robotics.in

#define BUZ 8
#define C4 262
#define D4 294
#define E4 330
#define F4 349
#define G4 392
#define A4 440

int melody[] = {C4,C4,G4,G4,A4,A4,G4,F4,F4,E4,E4,D4,D4,C4};
int beats[] = {1, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2};
```

```
void setup() {  
  pinMode(BUZ, OUTPUT);  
}  
  
void playNote(int freq, int dur) {  
  tone(BUZ, freq, dur * 200);  
  delay(dur * 220);  
  noTone(BUZ);  
}  
  
void loop() {  
  for (int i = 0; i < 14; i++) {  
    playNote(melody[i], beats[i]);  
  }  
  delay(2000); // Pause before repeating  
}
```

Pro Tips

Search for 'note frequency chart' to find frequencies for any musical note
Add a pushbutton so the melody only plays when pressed

Project 12 PIR Motion Detector Alarm

Difficulty: **Intermediate** | Category: **Sensor**

Description

Detect human motion using a Passive Infrared (PIR) sensor and trigger an audio-visual alarm. PIR sensors detect the IR radiation emitted by warm bodies.

Components Required

Arduino Uno, HC-SR501 PIR sensor, buzzer, red LED, green LED

Circuit Diagram

```
+-----+
| PIR SENSOR CIRCUIT |
+-----+
|           |
| PIR VCC -> 5V   |
| PIR GND -> GND   |
| PIR OUT -> Pin 2   |
|           |
| Pin 13 -> 220R -> RED LED-> GND (alarm)
| Pin 12 -> 220R -> GRN LED-> GND (ready)
| Pin 8 -> Buzzer -> GND |
+-----+
```

How It Works

The PIR sensor detects changes in IR radiation patterns. When a person (or warm object) moves through its field of view, the OUT pin goes HIGH for a programmable duration (set via onboard potentiometers).

Source Code

```
// Project 12: PIR Motion Detector Alarm
// @namma_robotics.in

int pirPin = 2;
int redLed = 13;
int grnLed = 12;
int buzPin = 8;

void setup() {
  pinMode(pirPin, INPUT);
  pinMode(redLed, OUTPUT);
  pinMode(grnLed, OUTPUT);
  pinMode(buzPin, OUTPUT);
  Serial.begin(9600);
}
```

```
// Sensor warm-up (important!)
Serial.println("PIR warming up... (30 sec)");
for (int i = 30; i > 0; i--) {
  Serial.print(i); Serial.println("s");
  delay(1000);
}
Serial.println("System ARMED");
digitalWrite(grnLed, HIGH);
}

void loop() {
  if (digitalRead(pirPin) == HIGH) {
    Serial.println("MOTION DETECTED!");
    digitalWrite(redLed, HIGH);
    digitalWrite(grnLed, LOW);
    tone(buzPin, 1500, 300);
    delay(300);
    tone(buzPin, 800, 300);
    delay(1000);
    digitalWrite(redLed, LOW);
    digitalWrite(grnLed, HIGH);
  }
}
```

Pro Tips

The two potentiometers on HC-SR501 adjust sensitivity and hold time
Allow a 30-60 second warm-up period for accurate detection

Project 13 DHT11 Weather Station

Difficulty: **Intermediate** | Category: [Sensor](#)

Description

Monitor temperature and humidity using the DHT11 digital sensor. A step up from analog sensors — the DHT11 uses a one-wire digital protocol.

Components Required

Arduino Uno, DHT11 sensor, 10kΩ pull-up resistor, LCD or Serial

Circuit Diagram

```
+-----+
| DHT11 CIRCUIT |
+-----+
|               |
| DHT11 Pin 1 (VCC) -> 5V |
| DHT11 Pin 2 (DATA) -> Pin 2 |
| DHT11 Pin 4 (GND) -> GND |
|               |
| Also: 10kR pull-up: |
| Pin 2 -> 10kR -> 5V |
+-----+
```

How It Works

The DHT11 uses a proprietary single-wire protocol. After the Arduino sends a start signal, the sensor responds with 40 bits of data: 8-bit humidity integer, 8-bit humidity decimal, 8-bit temp integer, 8-bit temp decimal, 8-bit checksum.

Source Code

```
// Project 13: DHT11 Temperature & Humidity
// Requires: DHT sensor library by Adafruit
// @namma_robotics.in

#include <DHT.h>

#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
  Serial.println("DHT11 Weather Station");
}
```

```
}

void loop() {
  delay(2000); // DHT11 max sample rate is 1 Hz

  float humidity = dht.readHumidity();
  float tempC    = dht.readTemperature();
  float tempF    = dht.readTemperature(true); // Fahrenheit

  if (isnan(humidity) || isnan(tempC)) {
    Serial.println("ERROR: Sensor read failed!");
    return;
  }

  Serial.print("Humidity: "); Serial.print(humidity); Serial.println(" %");
  Serial.print("Temperature: "); Serial.print(tempC);   Serial.println(" °C");
  Serial.print("          "); Serial.print(tempF);   Serial.println(" °F");
  Serial.println("---");
}
```

Pro Tips

For more accuracy use the DHT22, which has $\pm 0.5^{\circ}\text{C}$ vs DHT11's $\pm 2^{\circ}\text{C}$
Install the DHT library via Sketch > Include Library > Manage Libraries

Project 14 Keypad Password System

Difficulty: **Intermediate** | Category: **Input**

Description

Create a 4-digit password security system using a matrix keypad. Correct password unlocks (green LED), wrong password triggers alarm (red + buzzer).

Components Required

Arduino Uno, 4x4 matrix keypad, green LED, red LED, buzzer

Circuit Diagram

```
+-----+
| 4x4 KEYPAD CIRCUIT |
+-----+
|           |
| Keypad Row pins -> 9,8,7,6|
| Keypad Col pins -> 5,4,3,2|
|           |
| Pin 11 -> 220R -> GRN LED-> GND
| Pin 12 -> 220R -> RED LED-> GND
| Pin 10 -> Buzzer -> GND |
+-----+
```

How It Works

Matrix keypads work by scanning rows and columns. Each key press connects one row pin to one column pin. The Keypad library handles scanning and debouncing. '#' submits the entered password for checking.

Source Code

```
// Project 14: Keypad Password Lock
// Requires: Keypad library by Mark Stanley
// @namma_robotics.in

#include <Keypad.h>

const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  {'1','2','3','A'}, {'4','5','6','B'},
  {'7','8','9','C'}, {'*','0','#','D'}
};
byte rowPins[4] = {9, 8, 7, 6};
byte colPins[4] = {5, 4, 3, 2};
Keypad kpad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

```
String password = "1234";
String input  = "";
int grnLed = 11, redLed = 12, buzPin = 10;

void setup() {
  pinMode(grnLed, OUTPUT); pinMode(redLed, OUTPUT); pinMode(buzPin, OUTPUT);
  Serial.begin(9600);
  Serial.println("Enter password + #");
}

void loop() {
  char key = keypad.getKey();
  if (!key) return;
  if (key == '*') { input = ""; Serial.println("Cleared"); return; }
  if (key == '#') {
    if (input == password) {
      Serial.println("ACCESS GRANTED");
      digitalWrite(grnLed, HIGH); delay(2000); digitalWrite(grnLed, LOW);
    } else {
      Serial.println("ACCESS DENIED");
      digitalWrite(redLed, HIGH); tone(buzPin, 2000, 1000); delay(1500); digitalWrite(redLed, LOW); noTone(buzPin);
    }
    input = "";
  } else { input += key; Serial.println("*"); }
}
```

Pro Tips

Store the password in EEPROM so it survives power loss

Add an LCD to show prompts and asterisks for typed characters

Project 15 RFID Card Access Control

Difficulty: **Intermediate** | Category: **Wireless**

Description

Build a contactless card access system using the MFRC522 13.56MHz RFID reader. Register cards and grant/deny access based on unique card IDs.

Components Required

Arduino Uno, MFRC522 RFID reader, RFID cards/fobs, servo, green/red LEDs

Circuit Diagram

```
+-----+
| MFRC522 SPI CIRCUIT |
+-----+
|
| MFRC522 SDA -> Pin 10 (SS) |
| MFRC522 SCK -> Pin 13 (SCK) |
| MFRC522 MOSI -> Pin 11 (MOSI) |
| MFRC522 MISO -> Pin 12 (MISO) |
| MFRC522 RST -> Pin 9 |
| MFRC522 3.3V -> 3.3V |
| MFRC522 GND -> GND |
|
| Servo signal -> Pin 6 |
+-----+
```

How It Works

RFID cards contain a microchip and antenna. The MFRC522 reader emits a 13.56MHz RF field that powers the passive card and reads its unique 4-byte UID via the SPI interface. The sketch compares the UID to a whitelist.

Source Code

```
// Project 15: RFID Access Control
// Requires: MFRC522 library by GithubCommunity
// @namma_robotics.in

#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>

#define SS_PIN 10
#define RST_PIN 9
MFRC522 rfid(SS_PIN, RST_PIN);
Servo lockServo;
```

@namma_robotics.in

```
// Add your authorized card UIDs here
String authorizedUID = "A1B2C3D4"; // Change to your card's UID

int grnLed = 7, redLed = 6;

void setup() {
  SPI.begin(); rfid.PCD_Init();
  lockServo.attach(8);
  lockServo.write(0); // Locked position
  pinMode(grnLed, OUTPUT); pinMode(redLed, OUTPUT);
  Serial.begin(9600);
  Serial.println("Scan your card...");
}

void loop() {
  if (!rfid.PICC_IsNewCardPresent() || !rfid.PICC_ReadCardSerial()) return;

  String uid = "";
  for (byte i = 0; i < rfid.uid.size; i++)
    uid += String(rfid.uid.uidByte[i], HEX);
  uid.toUpperCase();

  Serial.print("Card UID: "); Serial.println(uid);

  if (uid == authorizedUID) {
    Serial.println("Access GRANTED");
    digitalWrite(grnLed, HIGH);
    lockServo.write(90); delay(3000); lockServo.write(0);
    digitalWrite(grnLed, LOW);
  } else {
    Serial.println("Access DENIED");
    digitalWrite(redLed, HIGH); delay(1000); digitalWrite(redLed, LOW);
  }
  rfid.PICC_HaltA();
}
```

Pro Tips

First run with `Serial.println(uid)` to find your card's UID

You can store multiple UIDs in a String array and loop through them

Project 16 Bluetooth LED Control

Difficulty: **Intermediate** | Category: **Wireless**

Description

Wirelessly control an LED from your smartphone via Bluetooth. Use any serial Bluetooth terminal app. A gateway project to IoT and wireless control.

Components Required

Arduino Uno, HC-05 Bluetooth module, LED, 220Ω resistor, voltage divider (1kΩ + 2kΩ for RX)

Circuit Diagram

```
+-----+
| HC-05 CIRCUIT |
+-----+
|           |
| HC-05 VCC -> 5V |
| HC-05 GND -> GND |
| HC-05 TX -> Pin 10 (RX)|
| HC-05 RX -> Voltage Divider -> Pin 11 (TX)|
| (1kR from Pin11, 2kR to GND)|
| (HC-05 RX is 3.3V tolerant) |
|           |
| Pin 13 -> 220R -> LED -> GND
+-----+
```

How It Works

The HC-05 acts as a serial bridge. When paired via Bluetooth, data sent from your phone app arrives on Pin 10 (SoftwareSerial RX). The sketch reads the character and controls the LED accordingly.

Source Code

```
// Project 16: Bluetooth LED Control
// App: "Serial Bluetooth Terminal" (Play Store/App Store)
// @namma_robotics.in

#include <SoftwareSerial.h>

SoftwareSerial bt(10, 11); // RX, TX
int ledPin = 13;

void setup() {
  Serial.begin(9600); // Debug
  bt.begin(9600); // HC-05 default baud
  pinMode(ledPin, OUTPUT);
  Serial.println("Bluetooth ready. Pair with HC-05.");
}
```

```
Serial.println("Send '1' to turn LED ON, '0' to turn OFF");
}

void loop() {
  if (bt.available()) {
    char cmd = bt.read();
    Serial.print("Received: "); Serial.println(cmd);

    if (cmd == '1') {
      digitalWrite(ledPin, HIGH);
      bt.println("LED is ON");
    } else if (cmd == '0') {
      digitalWrite(ledPin, LOW);
      bt.println("LED is OFF");
    } else {
      bt.println("Unknown command. Send 1 or 0.");
    }
  }
}
```

Pro Tips

Default HC-05 pairing password is '1234' or '0000'

Upgrade to HC-05 in AT mode to change baud rate and device name

Project 17 Stepper Motor Control

Difficulty: **Intermediate** | Category: **Motor**

Description

Control a stepper motor with precise step counts and directions. Stepper motors are used in 3D printers, CNC machines, and camera sliders.

Components Required

Arduino Uno, 28BYJ-48 stepper motor, ULN2003 driver board

Circuit Diagram

```
+-----+
| STEPPER MOTOR CIRCUIT |
+-----+
|           |
| ULN2003 IN1 -> Pin 8   |
| ULN2003 IN2 -> Pin 9   |
| ULN2003 IN3 -> Pin 10  |
| ULN2003 IN4 -> Pin 11  |
| ULN2003 VCC -> 5V      |
| ULN2003 GND -> GND     |
| Motor connector -> ULN2003 output
+-----+
```

How It Works

The 28BYJ-48 is a geared stepper (64 steps × 64:1 gear = 4096 steps/revolution in full-step mode). The ULN2003 driver handles the current amplification. The Stepper library sequences the coil energization automatically.

Source Code

```
// Project 17: Stepper Motor Control
// @namma_robotics.in

#include <Stepper.h>

// 28BYJ-48 full revolution = 2048 steps (half-step mode)
const int stepsPerRev = 2048;

// Pin order must match coil sequence: IN1, IN3, IN2, IN4
Stepper motor(stepsPerRev, 8, 10, 9, 11);

void setup() {
  motor.setSpeed(10); // RPM (max ~15 for 28BYJ-48)
```

```
Serial.begin(9600);
}

void loop() {
  Serial.println("Rotating clockwise one full revolution...");
  motor.step(stepsPerRev);
  delay(1000);

  Serial.println("Rotating counter-clockwise one full revolution...");
  motor.step(-stepsPerRev);
  delay(1000);

  Serial.println("Rotating 90 degrees clockwise...");
  motor.step(stepsPerRev / 4);
  delay(1000);
}
```

Pro Tips

The pin order 8,10,9,11 (not 8,9,10,11) is critical for correct rotation

For speed control, increase setSpeed() but the 28BYJ-48 tops out around 15 RPM

Project 18 Soil Moisture Auto-Watering

Difficulty: **Intermediate** | Category: [Automation](#)

Description

Build a plant watering system that automatically waters when soil is dry. A practical IoT project combining sensors, relays, and threshold logic.

Components Required

Arduino Uno, capacitive soil moisture sensor, relay module, small water pump, tubing

Circuit Diagram

```
+-----+
| AUTO WATERING CIRCUIT |
+-----+
|           |
| Sensor VCC -> 3.3V or 5V |
| Sensor GND -> GND      |
| Sensor AOUT -> A0       |
|           |
| Relay VCC  -> 5V        |
| Relay GND  -> GND       |
| Relay IN   -> Pin 7     |
| Relay COM  -> Pump +    |
| Relay NO   -> Power +   |
| Pump -     -> Power -   |
+-----+
```

How It Works

The capacitive moisture sensor measures soil dielectric permittivity (water content) without the corrosion issues of resistive sensors. The relay switches the pump's power circuit on/off safely, isolating the Arduino from the pump's voltage.

Source Code

```
// Project 18: Automatic Plant Watering System
// @namma_robotics.in

int moisturePin = A0;
int relayPin    = 7;

// Calibrate these values for your sensor
// Dry soil: ~600-800, Wet soil: ~200-400
int dryThreshold = 550;
int wetThreshold = 350;
```

```
bool pumpRunning = false;

void setup() {
  pinMode(relayPin, OUTPUT);
  digitalWrite(relayPin, HIGH); // Active-low relay: HIGH = pump OFF
  Serial.begin(9600);
  Serial.println("Auto-Watering System Ready");
}

void loop() {
  int moisture = analogRead(moisturePin);
  int percent = map(moisture, 800, 200, 0, 100);
  percent = constrain(percent, 0, 100);

  Serial.print("Moisture: "); Serial.print(percent); Serial.println("%");

  if (percent < 30 && !pumpRunning) {
    Serial.println("Soil DRY - Pump ON");
    digitalWrite(relayPin, LOW); // Pump ON
    pumpRunning = true;
  } else if (percent > 60 && pumpRunning) {
    Serial.println("Soil WET - Pump OFF");
    digitalWrite(relayPin, HIGH); // Pump OFF
    pumpRunning = false;
  }

  delay(5000); // Check every 5 seconds
}
```

Pro Tips

Calibrate by reading the sensor in completely dry soil (no water) and submerged in water
Add a water-level sensor in the reservoir to alert when water runs out

Project 19 DC Motor Speed & Direction

Difficulty: **Intermediate** | Category: **Motor**

Description

Control a DC motor's speed and direction using the L298N H-bridge driver. Introduces the H-bridge concept and bidirectional motor control.

Components Required

Arduino Uno, L298N H-Bridge module, DC motor, 9V battery, potentiometer

Circuit Diagram

```
+-----+
| L298N MOTOR CIRCUIT |
+-----+
|           |
| L298N ENA -> Pin 9 (PWM)|
| L298N IN1 -> Pin 7   |
| L298N IN2 -> Pin 8   |
| L298N VCC -> 9V battery |
| L298N GND -> Battery(-) + Arduino GND
| L298N 5V -> Arduino 5V (if jumper on)
| Motor A -> L298N OUT1 & OUT2
|           |
| Pot wiper -> A0      |
+-----+
```

How It Works

The L298N H-bridge uses four transistors to control motor polarity. IN1/IN2 set direction: HIGH/LOW = forward, LOW/HIGH = backward, both same = brake. ENA controls speed via PWM. The bridge handles up to 2A per channel.

Source Code

```
// Project 19: DC Motor Speed & Direction
// @namma_robotics.in

int enA = 9; // PWM speed control
int in1 = 7; // Direction pin 1
int in2 = 8; // Direction pin 2
int potPin = A0;

void forward(int spd) { digitalWrite(in1,HIGH); digitalWrite(in2,LOW); analogWrite(enA,spd); }
void backward(int spd) { digitalWrite(in1,LOW); digitalWrite(in2,HIGH); analogWrite(enA,spd); }
void stopMotor() { digitalWrite(in1,LOW); digitalWrite(in2,LOW); analogWrite(enA,0); }
```

```
void setup() {  
  pinMode(enA, OUTPUT); pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);  
  Serial.begin(9600);  
}  
  
void loop() {  
  int potVal = analogRead(potPin);  
  int speed = map(potVal, 0, 1023, 0, 255);  
  
  if (potVal > 550) {  
    forward(speed);  
    Serial.print("FORWARD speed="); Serial.println(speed);  
  } else if (potVal < 473) {  
    backward(255 - speed);  
    Serial.print("BACKWARD speed="); Serial.println(255-speed);  
  } else {  
    stopMotor();  
    Serial.println("STOPPED");  
  }  
  delay(100);  
}
```

Pro Tips

Always connect Arduino GND to L298N GND, even if using external power
The L298N gets hot at full load — add a heatsink to the chip

Project 20 IR Remote Control System

Difficulty: **Intermediate** | Category: **Wireless**

Description

Decode signals from any TV or appliance IR remote and use different buttons to control LEDs, relays, or servos. Understand IR modulation and the NEC protocol.

Components Required

Arduino Uno, TSOP1738 IR receiver, any IR remote, LEDs

Circuit Diagram

```
+-----+
| IR RECEIVER CIRCUIT |
+-----+
|           |
| TSOP1738 Pin 1 (OUT)-> Pin 11
| TSOP1738 Pin 2 (GND)-> GND |
| TSOP1738 Pin 3 (VCC)-> 5V |
| (100nF cap across VCC-GND) |
|           |
| Pin 13 -> 220R -> LED -> GND |
+-----+
```

How It Works

IR remotes transmit 38kHz modulated pulses. The TSOP receiver demodulates and outputs the raw pulse data. The IRremote library decodes protocols like NEC, Sony, RC5 and returns a unique hex code per button press.

Source Code

```
// Project 20: IR Remote Control
// Requires: IRremote library by shirriff
// @namma_robotics.in

#include <IRremote.h>

int recvPin = 11;
IRrecv irrecv(recvPin);
decode_results results;
int ledPin = 13;

// Record your remote's codes first, then fill these in:
#define BTN_POWER 0xFFA25D
#define BTN_ONE 0xFF6897
```

```
void setup() {  
  Serial.begin(9600);  
  irrecv.enableIRIn();  
  pinMode(ledPin, OUTPUT);  
  Serial.println("IR Receiver ready. Press any button.");  
}  
  
void loop() {  
  if (irrecv.decode(&results)) {  
    Serial.print("Code: 0x");  
    Serial.println(results.value, HEX);  
  
    if (results.value == BTN_POWER) {  
      Serial.println("Power button -> Toggle LED");  
      digitalWrite(ledPin, !digitalRead(ledPin));  
    } else if (results.value == BTN_ONE) {  
      Serial.println("Button 1 pressed!");  
    }  
  
    irrecv.resume(); // Ready for next signal  
  }  
}
```

Pro Tips

First run the sketch to discover codes, then hardcode them in your #define statements
0xFFFFFFFF means a repeat code (button held down)

Project 21 OLED Mini Display

Difficulty: **Intermediate** | Category: **Display**

Description

Drive a crisp 128x64 pixel OLED display over I2C. Display text, graphics, and animated data. OLED displays are brighter and lower power than LCDs.

Components Required

Arduino Uno, SSD1306 0.96" OLED display (I2C), jumper wires

Circuit Diagram

```
+-----+
| OLED I2C CIRCUIT |
+-----+
|                 |
| OLED VCC -> 3.3V or 5V |
| OLED GND -> GND      |
| OLED SCL -> Pin A5 (SCL)|
| OLED SDA -> Pin A4 (SDA)|
|                 |
| I2C address: 0x3C (most) |
| or 0x3D (some modules)  |
+-----+
```

How It Works

The SSD1306 controller drives $128 \times 64 = 8192$ pixels. I2C communication (SDA/SCL) transfers commands and bitmap data. The Adafruit_SSD1306 library manages the frame buffer and transfers it to the display on display() calls.

Source Code

```
// Project 21: OLED Display
// Requires: Adafruit SSD1306 + GFX libraries
// @namma_robotics.in

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_W 128
#define SCREEN_H 64
Adafruit_SSD1306 display(SCREEN_W, SCREEN_H, &Wire, -1);

void setup() {
```

```
Serial.begin(9600);
if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
  Serial.println("OLED not found!");
  while (1);
}
display.clearDisplay();

// Draw title
display.setTextSize(2);
display.setTextColor(SSD1306_WHITE);
display.setCursor(10, 5);
display.println("Namma");

// Draw subtitle
display.setTextSize(1);
display.setCursor(5, 35);
display.println("Robotics.in");
display.setCursor(5, 50);
display.println("Arduino Projects");

display.display();
}

void loop() {
  // Animated counter
  for (int i = 0; i <= 100; i++) {
    display.fillRect(0, 55, 128, 9, SSD1306_BLACK);
    display.setCursor(0, 56);
    display.print("Loading: "); display.print(i); display.print("%");
    display.display();
    delay(30);
  }
  delay(2000);
}
```

Pro Tips

Use `display.drawBitmap()` to show icons — convert images with LCD Assistant tool
I2C Scanner sketch can find your display's address if 0x3C doesn't work

Project 22 Smoke & Gas Detector

Difficulty: **Intermediate** | Category: **Safety**

Description

Detect smoke, LPG, methane, and CO using the MQ-2 electrochemical gas sensor. Build a safety alarm with threshold-based triggering.

Components Required

Arduino Uno, MQ-2 gas sensor, buzzer, red LED, green LED

Circuit Diagram

```
+-----+
| MQ-2 GAS SENSOR CIRCUIT |
+-----+
|           |
| MQ-2 VCC -> 5V      |
| MQ-2 GND -> GND      |
| MQ-2 AOUT -> A0      |
| MQ-2 DOUT -> Pin 2 (opt) |
|           |
| Pin 13 -> 220R -> RED LED-> GND
| Pin 12 -> 220R -> GRN LED-> GND
| Pin 8 -> Buzzer -> GND |
+-----+
```

How It Works

The MQ-2 contains a tin-dioxide (SnO_2) sensing element. In clean air, it has high resistance. In the presence of reducing gases (LPG, smoke, CO), resistance drops proportionally. A voltage divider converts this to an analog voltage read by A0.

Source Code

```
// Project 22: Smoke & Gas Detector
// @namma_robotics.in

int mq2Pin = A0;
int redLed = 13;
int grnLed = 12;
int buzPin = 8;
int threshold = 400; // Adjust based on calibration

void setup() {
  pinMode(redLed, OUTPUT);
  pinMode(grnLed, OUTPUT);
  pinMode(buzPin, OUTPUT);
}
```

```
Serial.begin(9600);

// Critical: sensor warm-up (at least 20 seconds)
Serial.print("MQ-2 warming up");
for (int i = 0; i < 20; i++) {
  Serial.print(".");
  delay(1000);
}
Serial.println("
Ready!");
digitalWrite(grnLed, HIGH);
}

void loop() {
  int gasLevel = analogRead(mq2Pin);
  Serial.print("Gas level: "); Serial.println(gasLevel);

  if (gasLevel > threshold) {
    Serial.println("*** ALARM: Gas/Smoke Detected! ***");
    digitalWrite(grnLed, LOW);
    digitalWrite(redLed, HIGH);
    tone(buzPin, 2500, 200); delay(250);
    tone(buzPin, 1000, 200); delay(250);
  } else {
    digitalWrite(redLed, LOW);
    digitalWrite(grnLed, HIGH);
    noTone(buzPin);
  }
  delay(500);
}
```

Pro Tips

New MQ sensors need 24-48 hours of power-on time to reach stable readings ('burn-in')

Check local safety regulations before using in a real alarm system

Project 23 7-Segment Display Counter

Difficulty: **Intermediate** | Category: [Display](#)

Description

Drive a 7-segment LED display to count from 0-9 by directly controlling individual segments. Understand segment encoding without using shift registers.

Components Required

Arduino Uno, common-cathode 7-segment display, 7x 220Ω resistors

Circuit Diagram

```
+-----+
| 7-SEGMENT CIRCUIT |
+-----+
| Segment a -> 220R -> Pin2 |
| Segment b -> 220R -> Pin3 |
| Segment c -> 220R -> Pin4 |
| Segment d -> 220R -> Pin5 |
| Segment e -> 220R -> Pin6 |
| Segment f -> 220R -> Pin7 |
| Segment g -> 220R -> Pin8 |
| Common cathode -> GND |
+-----+
```

How It Works

A 7-segment display has 7 LED segments (a-g) that form digit shapes when lit in combinations. Each combination is encoded as a byte. For example, digit '0' needs segments a,b,c,d,e,f lit (all except g).

Source Code

```
// Project 23: 7-Segment Display Counter
// @namma_robotics.in

// Pins for segments a,b,c,d,e,f,g
int segPins[] = {2, 3, 4, 5, 6, 7, 8};

// Segment patterns for 0-9 (a,b,c,d,e,f,g)
// 1 = segment ON, 0 = segment OFF
byte digits[10][7] = {
  {1,1,1,1,1,0}, // 0
  {0,1,1,0,0,0}, // 1
  {1,1,0,1,0,1}, // 2
  {1,1,1,1,0,0}, // 3
  {0,1,1,0,0,1}, // 4
  {1,0,1,1,0,1}, // 5
```

```
{1,0,1,1,1,1,1}, // 6
{1,1,1,0,0,0,0}, // 7
{1,1,1,1,1,1,1}, // 8
{1,1,1,1,0,1,1} // 9
};

void showDigit(int num) {
  for (int i = 0; i < 7; i++)
    digitalWrite(segPins[i], digits[num][i]);
}

void setup() {
  for (int i = 0; i < 7; i++) pinMode(segPins[i], OUTPUT);
}

void loop() {
  for (int d = 0; d <= 9; d++) {
    showDigit(d);
    delay(1000);
  }
}
```

Pro Tips

For common-anode displays, invert the logic (0 = ON, 1 = OFF)

Use a 74HC595 shift register to drive the display with only 3 Arduino pins

Project 24 Sound Level Meter

Difficulty: **Beginner** | Category: **Audio**

Description

Visualize sound levels as a bar graph of LEDs — just like a VU meter in audio equipment. Great introduction to analog signal processing.

Components Required

Arduino Uno, KY-038 microphone module, 5-10 LEDs, resistors

Circuit Diagram

```
+-----+
| SOUND METER CIRCUIT |
+-----+
|           |
| KY-038 VCC -> 5V   |
| KY-038 GND -> GND  |
| KY-038 A0  -> A0    |
|           |
| Pin 3 -> 220R -> LED1 -> GND
| Pin 4 -> 220R -> LED2 -> GND
| Pin 5 -> 220R -> LED3 -> GND
| Pin 6 -> 220R -> LED4 -> GND
| Pin 7 -> 220R -> LED5 -> GND
+-----+
```

How It Works

The KY-038 microphone amplifies sound and outputs a varying analog voltage (0–5V). The Arduino reads this and maps it to the number of LEDs to light. Louder sounds produce higher voltage, lighting more LEDs.

Source Code

```
// Project 24: Sound Level Meter VU Bar
// @namma_robotics.in

int micPin = A0;
int ledPins[] = {3, 4, 5, 6, 7};
int numLeds = 5;

void setup() {
  for (int i = 0; i < numLeds; i++)
    pinMode(ledPins[i], OUTPUT);
  Serial.begin(9600);
}
```

```
void loop() {  
  // Sample peak in a short window for smooth response  
  int peak = 0;  
  for (int i = 0; i < 10; i++) {  
    int val = analogRead(micPin);  
    if (val > peak) peak = val;  
    delay(5);  
  }  
  
  // Map peak to number of LEDs to light  
  int count = map(peak, 0, 1023, 0, numLeds);  
  Serial.println(peak);  
  
  for (int i = 0; i < numLeds; i++)  
    digitalWrite(ledPins[i], i < count ? HIGH : LOW);  
}
```

Pro Tips

Adjust the sensitivity potentiometer on the KY-038 module

Use green LEDs for low, yellow for mid, red for high (peak) levels

Project 25 Laser Tripwire Security

Difficulty: **Intermediate** | Category: **Security**

Description

Create an invisible security beam using a laser and LDR. When someone breaks the laser beam, the alarm triggers. Practical security project.

Components Required

Arduino Uno, 5mW laser module, LDR, 10kΩ resistor, buzzer, LED

Circuit Diagram

```
+-----+
| LASER TRIPWIRE CIRCUIT |
+-----+
|           |
| Laser(+) -> Pin 12   |
| Laser(-) -> GND     |
|           |
| [LDR] placed to receive |
| the laser beam       |
| 5V -> LDR -> A0      |
| A0 -> 10kR -> GND    |
|           |
| Pin 8 -> Buzzer -> GND |
| Pin 13-> 220R -> LED -> GND
+-----+
```

How It Works

The laser is aimed at the LDR across a doorway. In normal state, laser hits LDR (high light = high resistance drop = high A0 reading). When someone crosses, the beam breaks, A0 drops significantly, triggering the alarm.

Source Code

```
// Project 25: Laser Tripwire Security
// @namma_robotics.in

int laserPin = 12, ldrPin = A0, buzPin = 8, ledPin = 13;
int baseline = 0;
int tolerance = 150;

void setup() {
  pinMode(laserPin, OUTPUT);
  pinMode(buzPin, OUTPUT);
  pinMode(ledPin, OUTPUT);
}
```

```
Serial.begin(9600);

// Turn laser on
digitalWrite(laserPin, HIGH);
delay(2000); // Let LDR stabilize

// Calibrate: measure baseline with beam unbroken
baseline = analogRead(ldrPin);
Serial.print("Baseline: "); Serial.println(baseline);
Serial.println("System ARMED");
}

void loop() {
  int ldrVal = analogRead(ldrPin);

  if (ldrVal < baseline - tolerance) {
    // Beam broken!
    Serial.println("** INTRUDER DETECTED! **");
    digitalWrite(ledPin, HIGH);
    tone(buzPin, 3000, 150);
    delay(150);
    tone(buzPin, 1500, 150);
    delay(150);
  } else {
    digitalWrite(ledPin, LOW);
    noTone(buzPin);
  }
}
```

Pro Tips

Aim the laser carefully at the LDR center for maximum signal difference
Mount both laser and LDR firmly so vibrations don't cause false triggers

Project 26 Real-Time Clock Calendar

Difficulty: **Intermediate** | Category: [Display](#)

Description

Build a precision digital clock that keeps time even when Arduino is powered off, thanks to the DS3231's built-in battery backup.

Components Required

Arduino Uno, DS3231 RTC module, 16x2 LCD, jumper wires

Circuit Diagram

```
+-----+
| DS3231 RTC CIRCUIT |
+-----+
|                   |
| DS3231 VCC -> 5V (or 3.3V) |
| DS3231 GND -> GND      |
| DS3231 SCL -> A5 (SCL) |
| DS3231 SDA -> A4 (SDA) |
| (CR2032 coin cell on module)
|                   |
| LCD connections as per |
| Project 9             |
+-----+
```

How It Works

The DS3231 is a temperature-compensated crystal oscillator (TCXO) RTC with I2C interface. Its internal oscillator is accurate to ± 2 ppm (about 1 minute per year). The onboard CR2032 battery keeps time when Arduino is unpowered.

Source Code

```
// Project 26: Real-Time Clock with LCD
// Requires: RTCLib by Adafruit
// @namma_robotics.in

#include <Wire.h>
#include <RTCLib.h>
#include <LiquidCrystal.h>

RTC_DS3231 rtc;
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

char daysOfWeek[7][10] = {
```

```
"Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"
};

void setup() {
  Serial.begin(9600);
  lcd.begin(16, 2);
  Wire.begin();

  if (!rtc.begin()) {
    lcd.print("RTC not found!");
    while (1);
  }

  // Uncomment ONCE to set time, then re-comment and re-upload:
  // rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
}

void loop() {
  DateTime now = rtc.now();

  // Row 0: Time
  lcd.setCursor(0, 0);
  if (now.hour() < 10) lcd.print("0"); lcd.print(now.hour()); lcd.print(":");
  if (now.minute() < 10) lcd.print("0"); lcd.print(now.minute()); lcd.print(":");
  if (now.second() < 10) lcd.print("0"); lcd.print(now.second());
  lcd.print(" ");

  // Row 1: Date
  lcd.setCursor(0, 1);
  if (now.day() < 10) lcd.print("0"); lcd.print(now.day()); lcd.print("/");
  if (now.month() < 10) lcd.print("0"); lcd.print(now.month()); lcd.print("/");
  lcd.print(now.year());

  delay(1000);
}
```

Pro Tips

Run the time-set line once, re-comment it, and re-upload so the clock doesn't reset on each power cycle

DS3231 is more accurate than DS1307 and includes a built-in temperature sensor

Project 27 Joystick Servo Controller

Difficulty: **Beginner** | Category: **Control**

Description

Control two servo motors (pan + tilt) with an analog joystick. The foundation of a camera gimbal, robotic arm, or pan-tilt turret.

Components Required

Arduino Uno, analog joystick module, 2x servo motors

Circuit Diagram

```
+-----+
| JOYSTICK + SERVOS |
+-----+
|           |
| Joystick VCC -> 5V |
| Joystick GND -> GND |
| Joystick VRx -> A0 |
| Joystick VRy -> A1 |
| Joystick SW -> Pin 2 |
|           |
| Servo X signal -> Pin 9 |
| Servo Y signal -> Pin 10 |
| Both Servo VCC -> 5V |
| Both Servo GND -> GND |
+-----+
```

How It Works

The joystick contains two potentiometers (X and Y axes) and a momentary pushbutton. Reading both analog values and mapping them to 0°–180° gives smooth, intuitive servo control. The button can be used for a center/reset function.

Source Code

```
// Project 27: Joystick Servo Controller (Pan-Tilt)
// @namma_robotics.in

#include <Servo.h>

Servo servoX, servoY;
int joyX = A0, joyY = A1, joyBtn = 2;

void setup() {
  servoX.attach(9);
```

```
servoY.attach(10);
pinMode(joyBtn, INPUT_PULLUP);
Serial.begin(9600);
}

void loop() {
  int xVal = analogRead(joyX);
  int yVal = analogRead(joyY);

  int angleX = map(xVal, 0, 1023, 0, 180);
  int angleY = map(yVal, 0, 1023, 0, 180);

  servoX.write(angleX);
  servoY.write(angleY);

  Serial.print("X:"); Serial.print(angleX);
  Serial.print(" Y:"); Serial.println(angleY);

  // Button press: center both servos
  if (digitalRead(joyBtn) == LOW) {
    servoX.write(90); servoY.write(90);
    Serial.println("CENTERED");
    delay(300);
  }

  delay(20);
}
```

Pro Tips

Add deadzone logic: if (abs(xVal - 512) < 50) angleX = 90; to prevent servo jitter at center
Mount servos on a 3D-printed pan-tilt bracket for a camera tracker

Project 28 Water Level Alarm

Difficulty: **Beginner** | Category: **Sensor**

Description

Monitor water level in a tank with visual indicators and an audio alarm when it's full or dangerously high. Useful for overhead tanks and flood detection.

Components Required

Arduino Uno, water level sensor module, LEDs (green/yellow/red), buzzer

Circuit Diagram

```
+-----+
| WATER LEVEL CIRCUIT |
+-----+
|           |
| Sensor S (signal) -> A0 |
| Sensor + (VCC) -> 5V |
| Sensor - (GND) -> GND |
|           |
| Pin 9 -> 220R -> GRN -> GND
| Pin 10 -> 220R -> YLW -> GND
| Pin 11 -> 220R -> RED -> GND
| Pin 8 -> Buzzer -> GND |
+-----+
```

How It Works

The resistive water sensor has exposed traces. Water between the traces creates a partial circuit, lowering resistance proportional to submersion depth. The analog reading rises as more of the sensor is submerged.

Source Code

```
// Project 28: Water Level Alarm
// @namma_robotics.in

int sensorPin = A0;
int grnLed = 9, ylwLed = 10, redLed = 11, buzPin = 8;

void setup() {
  pinMode(grnLed, OUTPUT); pinMode(ylwLed, OUTPUT);
  pinMode(redLed, OUTPUT); pinMode(buzPin, OUTPUT);
  Serial.begin(9600);
}
```

```
void loop() {  
  int level = analogRead(sensorPin);  
  Serial.print("Water Level: "); Serial.println(level);  
  
  // Turn all off first  
  digitalWrite(grnLed, LOW); digitalWrite(ylwLed, LOW);  
  digitalWrite(redLed, LOW); noTone(buzPin);  
  
  if (level < 200) {  
    Serial.println("Level: EMPTY");  
    // All LEDs off - tank empty  
  } else if (level < 400) {  
    Serial.println("Level: LOW");  
    digitalWrite(grnLed, HIGH);  
  } else if (level < 600) {  
    Serial.println("Level: MEDIUM");  
    digitalWrite(grnLed, HIGH); digitalWrite(ylwLed, HIGH);  
  } else if (level < 800) {  
    Serial.println("Level: HIGH");  
    digitalWrite(grnLed, HIGH); digitalWrite(ylwLed, HIGH); digitalWrite(redLed, HIGH);  
  } else {  
    Serial.println("Level: FULL - ALARM!");  
    digitalWrite(redLed, HIGH);  
    tone(buzPin, 1500); // Continuous alarm  
  }  
  
  delay(500);  
}
```

Pro Tips

Resistive sensors corrode over time in water — capacitive sensors last longer

Add a relay to cut power to a pump when the tank is full

Project 29 Digital Dice

Difficulty: **Beginner** | Category: **Fun**

Description

Press a button to roll a virtual dice. LEDs arranged in dice dot positions randomly show 1-6. A fun project for board games or teaching random number generation.

Components Required

Arduino Uno, pushbutton, 7 LEDs, 7x 220Ω resistors

Circuit Diagram

```
+-----+
| DICE LED POSITIONS |
+-----+
| [d1]  [d2]  |
|  [d3]      |
| [d4] [c] [d5] |
|  [d6]      |
| [d7]  [d8...skip] |
|              |
| d1:Pin3 d2:Pin4  |
| d3:Pin5 (center) |
| d4:Pin6 d5:Pin7  |
| d6:Pin8 d7:Pin9  |
| Button: Pin 2    |
+-----+
```

How It Works

Seven LEDs represent the seven possible dot positions on a dice face. Each number 1-6 lights a specific subset of these positions. `random()` generates a fair 1-6 roll, and the corresponding LED pattern is displayed.

Source Code

```
// Project 29: Digital Dice
// @namma_robotics.in

int btnPin = 2;
// LED pins for 7 dot positions
int dots[] = {3, 4, 5, 6, 7, 8, 9};

// Which dots light for each face (0=off, 1=on)
// Index: [face-1][dot 0-6]
int face[6][7] = {
    {0,0,1,0,0,0,0}, // 1: center only
```

```
{1,0,0,0,0,0,1}, // 2: top-right, bottom-left
{1,0,1,0,0,0,1}, // 3: top-right, center, bottom-left
{1,1,0,0,0,1,1}, // 4: four corners
{1,1,1,0,0,1,1}, // 5: four corners + center
{1,1,1,1,1,1,0} // 6: all except center
};

void showFace(int f) {
  for (int i = 0; i < 7; i++)
    digitalWrite(dots[i], face[f][i]);
}

void setup() {
  for (int i = 0; i < 7; i++) pinMode(dots[i], OUTPUT);
  pinMode(btnPin, INPUT_PULLUP);
  randomSeed(analogRead(A0)); // Seed with noise
  Serial.begin(9600);
}

void loop() {
  if (digitalRead(btnPin) == LOW) {
    // Quick rolling animation
    for (int i = 0; i < 10; i++) {
      showFace(random(0, 6));
      delay(80 + i * 20); // Slow down gradually
    }
    int roll = random(0, 6);
    showFace(roll);
    Serial.print("Rolled: "); Serial.println(roll + 1);
    delay(500); // Debounce
  }
}
```

Pro Tips

The 'rolling animation' makes it feel more like a real dice throw

Add a 7-segment display to show the number alongside the dot display

Project 30 Line Following Robot

Difficulty: **Advanced** | Category: **Robotics**

Description

Build an autonomous robot that follows a black line on a white surface. The fundamental robotics project covering sensor fusion and differential drive control.

Components Required

Arduino Uno, 2x IR line sensors (TCRT5000), L298N module, 2x DC motors + wheels, chassis, battery

Circuit Diagram

```
+-----+
| LINE FOLLOWER CIRCUIT |
+-----+
|
| Left Sensor OUT -> Pin 2|
| Right Sensor OUT -> Pin 3|
| Both Sensors VCC -> 5V |
| Both Sensors GND -> GND |
|
| L298N ENA -> Pin 9 (PWM)|
| L298N IN1 -> Pin 5 |
| L298N IN2 -> Pin 6 |
| L298N ENB -> Pin 10 (PWM)|
| L298N IN3 -> Pin 7 |
| L298N IN4 -> Pin 8 |
|
| Motor A -> L298N OUT1/2 |
| Motor B -> L298N OUT3/4 |
+-----+
```

How It Works

IR sensors detect the difference in reflectivity between black (absorbs IR) and white (reflects IR). When both sensors see white, drive forward. When left sees black, turn left. When right sees black, turn right. When both see black, stop.

Source Code

```
// Project 30: Line Following Robot
// @namma_robotics.in

// Motor A (Left)
int enA = 9, in1 = 5, in2 = 6;
// Motor B (Right)
int enB = 10, in3 = 7, in4 = 8;
```

@namma_robotics.in

```
// Sensors (LOW = black line detected)
int leftSensor = 2, rightSensor = 3;

int baseSpeed = 180;

void forward() { analogWrite(enA,baseSpeed); analogWrite(enB,baseSpeed); digitalWrite(in1,HIGH);
digitalWrite(in2,LOW); digitalWrite(in3,HIGH); digitalWrite(in4,LOW); }
void turnLeft() { analogWrite(enA,100); analogWrite(enB,baseSpeed); digitalWrite(in1,HIGH); digitalWrite(in2,LOW);
digitalWrite(in3,HIGH); digitalWrite(in4,LOW); }
void turnRight() { analogWrite(enA,baseSpeed); analogWrite(enB,100); digitalWrite(in1,HIGH); digitalWrite(in2,LOW);
digitalWrite(in3,HIGH); digitalWrite(in4,LOW); }
void stopMotors() { analogWrite(enA,0); analogWrite(enB,0); }

void setup() {
  pinMode(enA,OUTPUT); pinMode(in1,OUTPUT); pinMode(in2,OUTPUT);
  pinMode(enB,OUTPUT); pinMode(in3,OUTPUT); pinMode(in4,OUTPUT);
  pinMode(leftSensor, INPUT); pinMode(rightSensor, INPUT);
  Serial.begin(9600);
}

void loop() {
  int L = digitalRead(leftSensor); // LOW = on line
  int R = digitalRead(rightSensor);

  if (L == HIGH && R == HIGH) { forward(); } // Both on white: go forward
  else if (L == LOW && R == HIGH) { turnLeft(); } // Left on line: turn left
  else if (L == HIGH && R == LOW) { turnRight(); } // Right on line: turn right
  else
    { stopMotors(); } // Both on line: stop/at end
}
```

Pro Tips

Adjust baseSpeed for your motors; too fast causes the robot to overshoot the line

For smoother following, implement PID control using analog sensors instead of digital

Thank You!

*We hope this ebook sparked your curiosity and gave you the confidence
to build amazing things with Arduino.*

Every expert was once a beginner. Keep experimenting, keep learning, and never be afraid to break things — that is how the best inventions happen.

Connect With Namma Robotics

- Website: namma_robotics.in
- Instagram: [@namma_robotics.in](https://www.instagram.com/namma_robotics.in)
- YouTube: Search 'Namma Robotics' for video tutorials
- Email: hello@namma_robotics.in

Continue Your Learning Journey

- Explore Arduino Nano and ESP32 for wireless IoT projects
- Learn PCB design with KiCad to make your projects permanent
- Try MicroPython on Raspberry Pi Pico for Python-based hardware
- Join robotics competitions — apply what you have built here!
- Follow our intermediate ebook: 'Arduino IoT Projects' (coming soon)

Share the Knowledge

Did this ebook help you? Share it with a friend who wants to learn electronics!

Tag us [@namma_robotics.in](https://www.instagram.com/namma_robotics.in) with your project builds — we love seeing what you create.

Leave a review on our website — it helps other learners find this resource.

Happy Making!

— *Team Namma Robotics*

Copyright © 2026 Namma Robotics | @namma_robotics.in